

C 언어 포인터 복습 보고서

ROBIT 21기 수습단원 박도연

목차

1. 포인터의 기본 개념
2. 포인터와 함수
3. 배열과 포인터
4. 배열과 함수 및 배열의 크기
5. 포인터와 문자열
6. 이중 포인터
7. 학습 내용 정리 및 느낀 점

1. 포인터의 기본 개념

포인터는 다른 변수의 메모리 주소를 저장하는 변수이다. 일반 변수에는 정수나 문자와 같은 실제 값이 저장되지만, 포인터 변수에는 해당 변수가 저장된 위치가 저장된다.

```
int num = 10;
```

```
int* p = &num;
```

위 코드에서 num에는 정수 10이 저장되고, 포인터 변수 p에는 num의 주소가 저장된다.

& 연산자는 변수의 주소를 구하고, * 연산자는 포인터가 가리키는 주소에 저장된 값에 접근할 때 사용한다.

```
printf("%d\n", num);
```

```
printf("%d\n", *p);
```

p가 num을 가리키기 때문에 num과 *p는 모두 10을 출력한다. 또한 다음과 같이 포인터를 이용해 원래 변수의 값을 변경할 수도 있다.

```
*p = 20;
```

이 문장이 실행되면 p가 가리키는 변수인 num의 값이 20으로 변경된다.

포인터와 변수



p에는 num의 주소(1000)가 저장되어 있다.

2. 포인터와 함수

일반 변수를 함수에 전달하면 변수의 값이 복사되어 전달된다. 따라서 함수 안에서 값을 변경해도 원래 변수에는 영향을 주지 않는다.

반면 변수의 주소를 함수에 전달하면 포인터를 이용하여 원본 변수의 값을 변경할 수 있다.

```
#include <stdio.h>
```

```
void changeValue(int* p);
```

```
int main(void)
{
    int num = 10;

    printf("변경 전: %d\n", num);

    changeValue(&num);

    printf("변경 후: %d\n", num);

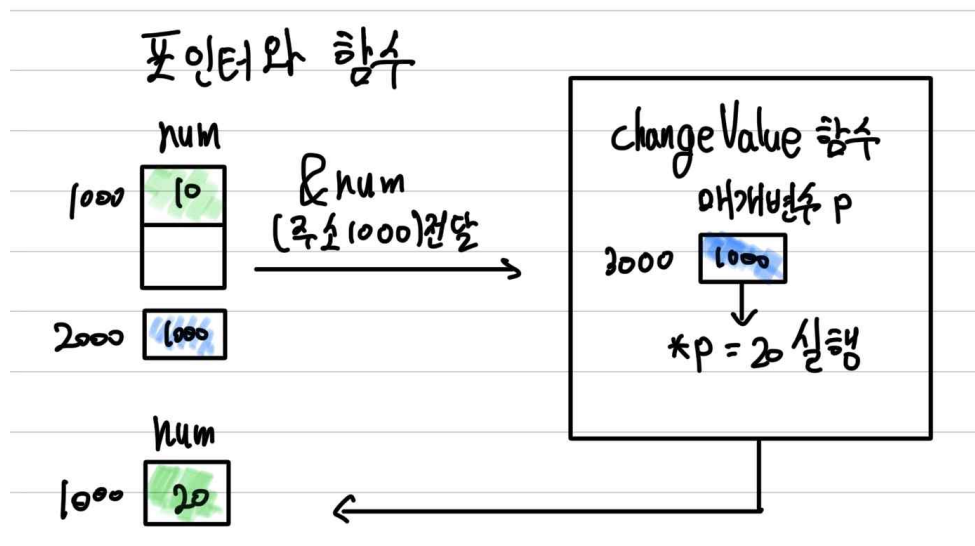
    return 0;
}
```

```
void changeValue(int* p)
{
    *p = 20;
}
```

실행 결과는 다음과 같다.

```
변경 전 : 10
변경 후 : 20
```

changeValue(&num)에서 num의 주소를 함수에 전달한다. 함수의 포인터 변수 p는 이 주소를 저장하고, *p = 20을 통해 원본 변수의 값을 변경하므로, 함수 밖에 있는 변수의 값을 변경해야 할 때는 변수의 주소를 전달하고 포인터 매개변수를 사용해야 한다.



3. 배열과 포인터

배열의 이름은 대부분의 표현식에서 배열의 첫 번째 요소의 주소처럼 사용된다.

```
int list[3] = { 10, 20, 30 };
```

위 배열에서 list와 &list[0]은 같은 시작 주소를 나타낸다.

배열의 요소는 인덱스 방식과 포인터 방식으로 모두 접근할 수 있다.

```
list[0] == *(list + 0)
```

```
list[1] == *(list + 1)
```

```
list[2] == *(list + 2)
```

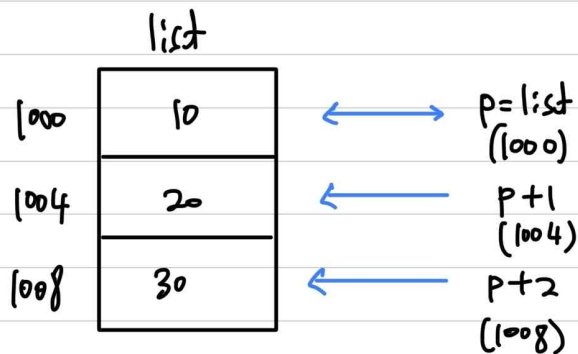
포인터에 정수를 더하면 다음 배열 요소의 주소로 이동한다. int형이 4바이트인 환경에서는 list + 1이 첫 번째 주소에서 4바이트 뒤의 주소를 가리킨다.

```
printf("%d\n", *(list + 1));
```

위 코드는 list[1]의 값인 20을 출력한다.

포인터끼리 뺄 때는 두 포인터 사이에 몇 개의 배열 요소가 있는지를 구할 수 있다. 다만 포인터끼리 더하거나 곱셈, 나눗셈 등의 연산을 할 수는 없다.

포인터와 산술



p에 1을 더하면 인덱스 2가 (4바이트)만큼

다음 배열 요소의 주소로 이동

4. 배열과 함수 및 배열의 크기

배열을 함수에 전달하면 배열 전체가 복사되는 것이 아니라 배열의 시작 주소가 전달된다.

```
void printArray(int* array, int size)
{
    int i;

    for (i = 0; i < size; i++)
    {
        printf("%d ", array[i]);
    }
}
```

함수는 배열의 시작 주소만 전달받기 때문에 배열의 길이를 자동으로 알 수 없다. 따라서 배열과 함께 요소의 개수도 전달해야 한다.

```
int list[3] = { 10, 20, 30 };
```

```
printArray(list, 3);
```

배열이 선언된 곳에서는 sizeof 연산자를 이용해 배열의 요소 개수를 구할 수 있다.

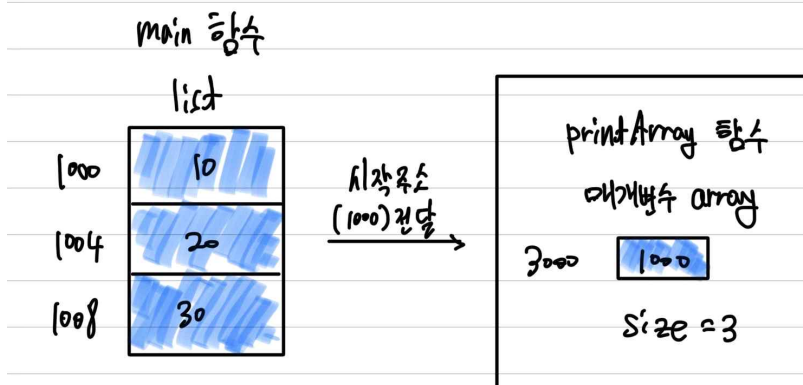
```
int size = sizeof(list) / sizeof(list[0]);
```

sizeof(list)는 배열 전체의 크기이고, sizeof(list[0])은 요소 하나의 크기이다. 전체 크기를 요소 하나의 크기로 나누면 배열의 요소 개수를 구할 수 있다.

함수에 전달된 배열은 포인터로 처리되므로 함수 안에서 sizeof(array)를 사용해 원래 배열의 크기를 구할 수 없으므로 배열의 길이는 별도의 매개변수로 전달해야 한다.

또한 배열의 범위를 벗어난 인덱스에 접근하면 잘못된 메모리를 읽거나 수정할 수 있으므로 반복문의 범위를 배열 길이에 맞게 설정해야 한다.

배열과 함수



함수는 배열의 시작 주소 (1000)만 전달받을
따라서 배열의 길이 size도 전달해야 함

5. 포인터와 문자열

C 언어에서 문자열은 여러 문자를 연속해서 저장한 문자 배열이다. 문자열의 마지막에는 문자열의 끝을 나타내는 널 종료 문자 '\0'이 저장된다.

```
char str[] = "Hello";
```

실제 배열에는 다음과 같이 저장된다.

```
'H' 'e' 'l' 'l' 'o' '\0'
```

문자열은 문자 배열과 문자 포인터의 형태로 나타낼 수 있다.

```
char s1[] = "Hello";  
const char* s2 = "Hello";
```

s1은 문자 배열이므로 다음과 같이 개별 문자를 변경할 수 있다.

```
s1[0] = 'h';
```

s2는 문자열 리터럴의 주소를 저장하는 포인터이다. 문자열 리터럴은 직접 수정하면 안 되므로 const char*로 선언하는 것이 안전하다.

문자 배열은 처음 정한 크기가 고정되지만, 문자 포인터는 다른 문자열을 가리키도록 변경할 수 있다.

```
s2 = "Pointer";
```

6. 이중 포인터

이중 포인터는 포인터 변수의 주소를 저장하는 포인터이다.

```
int num = 10;  
int* p = &num;  
int** pp = &p;
```

p에는 num의 주소가 저장되고, pp에는 포인터 변수 p의 주소가 저장된다.

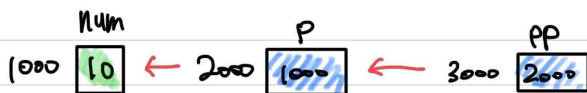
```
printf("%d\n", num);  
printf("%d\n", *p);  
printf("%d\n", **pp);
```

세 출력 결과는 모두 10이다.

*p는 p가 가리키는 변수 num의 값을 의미한다. *pp는 포인터 변수 p를 의미하며, **pp는 p가 가리키는 num의 값에 접근한다.

이중 포인터는 함수에서 외부 포인터가 가리키는 위치 자체를 변경해야 할 때 사용할 수 있다.

이중 포인터



*p → 10 p는 num의 주소를 저장

**p → 10 pp는 p의 주소를 저장

7. 학습 내용 정리 및 느낀 점

이번 과제를 통해 포인터는 다른 변수의 값을 저장하는 것이 아니라 메모리 주소를 저장하는 변수라는 것을 알게 되었다. 처음에는 *가 포인터를 선언할 때와 값을 참조할 때 모두 사용되어 헷갈렸지만, 포인터가 가리키는 값을 의미한다고 생각하니 이해하기 쉬웠다.

또한 배열의 이름이 첫 번째 요소의 주소처럼 사용되기 때문에 배열과 포인터가 밀접하게 연결되어 있다는 것을 알게 되었다. 배열을 함수에 전달하면 시작 주소만 전달되기 때문에 배열의 길이를 함께 전달해야 하는 이유도 이해할 수 있었다.

문자열도 문자 배열로 저장되며 마지막에는 '\0'이 들어간다는 점을 복습했다. 이중 포인터는 아직 어렵게 느껴지지만, 포인터의 주소를 저장하는 포인터라는 기본 구조는 이해할 수 있었다.